LAB 6 – CODE AND BRANCH COVERAGE ANALYSIS

Student Name: Pooja L R
Experiment: Lab 6 – Code and Branch Coverage Analysis
Project: Loan Eligibility System
Programming Language: Python 3.13.14
Testing Framework: Pytest
Coverage Tool: Coverage.py

---

1. Objective

The objective of this experiment is to understand and implement code coverage and branch coverage analysis using Python testing tools.

The experiment aims to:

- Understand the difference between code coverage and branch coverage.
- Measure the percentage of application code executed during testing.
- Identify untested statements and decision branches.
- Develop comprehensive test cases to achieve at least 90% coverage.
- Analyse coverage reports and improve test completeness.

2. Tools and Technologies Used

- Python 3.13.14
- Pytest 9.2.0
- Coverage.py 7.17.2
- PowerShell
- Git and GitHub

3. Project Description

The Loan Eligibility System is a Python application that determines whether an applicant qualifies for a loan based on employment type, age, income and credit score.

The application consists of three main functions:

1. credit_band(score)

Classifies an applicant's credit score into four categories:

- Excellent
- Good
- Fair

- Poor

It also raises a ValueError for invalid credit scores outside the permitted range.

2. is_eligible(age, income, employment, score)

Determines loan eligibility based on:

- Valid employment type.
- Age between 21 and 60.
- Minimum income requirements for the employment category.
- Credit score classification.

3. interest_rate(score, employment, has_collateral)

Calculates the applicable interest rate based on the applicant's credit band, employment type and collateral availability.

4. Initial Coverage Analysis

Initially, the existing test file contained only two test cases:

- Testing the excellent credit score category.
- Testing a successful loan eligibility scenario.

The initial test cases were executed using Pytest and both tests passed.

However, the initial coverage analysis revealed that several statements and decision branches were not executed.

Metric	Initial Result
Application statement coverage	36%
Application branch coverage	32%
Existing test cases	2

The results indicated that additional test cases were required to cover different credit score categories, invalid inputs, eligibility conditions and interest rate calculations.

5. Code Coverage Implementation

A separate test file named CodeCoverageTest_loan_eligibility.py was created.

The test cases covered:

- All four credit score categories.
- Invalid credit score handling.
- Unknown employment type.

- Age-based rejection.
- Income-based rejection.
- Rejection due to poor credit score.
- Successful loan eligibility.
- Interest rate calculation for excellent and good credit bands.
- Interest rate calculation with self-employment and collateral.

The following command was used to execute the tests and measure code coverage:

python -m coverage run --source=loan_eligibility -m pytest CodeCoverageTest_loan_eligibility.py -v

The coverage report was generated using:

python -m coverage report -m

Code Coverage Results

Metric	**Result**
Total test cases	10
Tests passed	10
Statements	33
Missed statements	0
Code coverage	100%

All executable statements in the application were successfully executed during testing.

Screenshot 1: Code_Coverage_Result.png.jpeg

6. Branch Coverage Implementation

After achieving complete code coverage, a separate test file named BranchCoverageTest_loan_eligibility.py was created.

Branch coverage was used to verify that both outcomes of conditional statements were executed.

The test cases covered:

- Credit scores below the minimum and above the maximum.
- Valid credit score boundaries.
- Known and unknown employment types.

- Age below, within and above the permitted range.
- Income below the minimum for salaried and self-employed applicants.
- Successful eligibility for different employment categories.
- Poor and good credit score eligibility decisions.
- Excellent, good, fair and poor credit bands in interest rate calculations.
- Interest rate adjustments for self-employment and collateral.

The following command was used to execute the tests and measure branch coverage:

python -m coverage run --branch --source=loan_eligibility -m pytest BranchCoverageTest_loan_eligibility.py -v

The report was generated using:

python -m coverage report -m

Branch Coverage Results

Metric	**Result**
Total test cases	16
Tests passed	16
Statements	33
Missed statements	0
Total branches	24
Partial branches	0
Branch coverage	100%

All 24 branches were successfully covered, with no partially covered branches.

Screenshot 2: Branch_Coverage_Result.png.jpeg

7. Comparison of Initial and Final Results

Coverage Metric	**Initial**	**Final**
Code coverage	36%	100%
Branch coverage	32%	100%

The additional test cases significantly improved the test completeness of the application.

The final results demonstrate that all executable statements and all measured decision branches in the application were covered.

8. Git and Version Control

The updated test files and corresponding screenshots were committed separately to maintain a clear development history.

The following commits were created:

Commit 1 – Code Coverage

9ba3974

Message: test: add comprehensive code coverage tests

Commit 2 – Branch Coverage

d9ff693

Message: test: add comprehensive branch coverage tests

Both commits were successfully pushed to the student's GitHub fork.

GitHub Repository:

https://github.com/pooja-lr/L6_L7_26_loan_eligibility

9. Learning Outcomes

Through this experiment, the following concepts were understood and implemented:

1. Code coverage measures the proportion of executable statements exercised by test cases.
2. Branch coverage measures whether different decision outcomes are executed.
3. Achieving 100% code coverage does not automatically guarantee 100% branch coverage.
4. Boundary-value testing and negative testing help identify untested conditions.
5. Coverage reports help identify missing statements and branches.
6. Git commits help maintain a structured history of testing improvements.

10. Conclusion

The Code and Branch Coverage Analysis experiment was successfully completed using Python, Pytest and Coverage.py.

Initially, the application achieved 36% code coverage and 32% branch coverage. Additional test cases were developed to exercise the different functionalities, input conditions, boundary values and decision branches of the Loan Eligibility System.

After implementing the additional tests, the application achieved **100% code coverage and 100% branch coverage**.

A total of 26 test cases were successfully executed across the two test files, with all tests passing.

Thus, the experiment successfully demonstrated the importance of code coverage and branch coverage in evaluating software testing completeness and identifying untested execution paths.